

UNIVERSITY OF MINES AND TECHNOLOGY (UMAT), TARKWA, GHANA.

FACULTY OF COMPUTING AND MATHEMATICAL SCIENCES

(FCaMS)

DEPARTMENT OF CYBER SECURITY AND INFORMATION SYSTEM



POST-ENGAGEMENT DEBRIEF AND REMEDIATION TRACKING REPORT

A Controlled Lab Engagement Against a Vulnerable Web Application

BY

DAMIAN SEYRAM SOSAVI

Index Number: FCM.41.018.243.23

Team: Red Team

Course Code: CY376 — Network Monitoring, Security and Auditing

Lecturer: Dr Frederick Edem Broni

Submission Date: 3rd August 2026

GitHub Repository: [<https://github.com/seyransosavi-stack/cy376-dvwa-red-team-project.git>]

Acknowledgements

I'd like to thank my lecturer and my institution for the guidance and the structure that made it possible to turn a small lab exercise into a full engagement lifecycle. I'd also like to thank anyone who reviewed drafts of this report and pushed back on the parts that weren't good enough yet — that feedback is the reason the final version holds up under scrutiny.

Abstract

This report documents a controlled red team engagement against a deliberately vulnerable web application (DVWA), deployed inside an isolated Docker container with its security level set to Low to expose the maximum realistic attack surface. Three vulnerability classes were tested through manual exploitation: SQL injection, OS command injection, and brute-force authentication. The SQL injection test achieved a full authentication logic bypass and extracted the entire user table, including password hashes — two of which were found to be identical, confirming the use of unsalted MD5 hashing. The command injection test achieved arbitrary command execution on the underlying host, confirmed at the web-server privilege level (www-data). The brute-force test confirmed the application enforces no lockout, delay, or CAPTCHA on repeated login attempts and transmits credentials via GET requests. Each finding was mapped to its relevant MITRE ATT&CK technique, carried through a structured post-engagement debrief, and converted into a remediation tracking framework that assigns severity, ownership, and a fixed deadline to every finding, alongside a formal retest-and-closure workflow. The overall conclusion is that vulnerability identification alone creates no measurable security value — an assessment only becomes useful once its findings are converted into owned, timed, and independently verified remediation work.

Table of Contents

(Right-click this field in Word and select 'Update Field' once the document is finalized, so page numbers populate correctly.)

List of Figures

Figure 1: Docker version check and initial container pull command

Figure 2: Docker image layers pulling successfully

Figure 3: MariaDB and Apache services starting inside the container

Figure 4: Apache server logs confirming successful startup

Figure 5: DVWA login page

Figure 6: DVWA home page describing the application's purpose

Figure 7: DVWA Security Level page set to Low

Figure 8: Command injection payload (127.0.0.1 && whoami) returning ping output plus www-data

Figure 9: Baseline SQL query — normal input (ID: 1) returning single expected user

Figure 10: Logic bypass payload (1' OR '1'='1' -- -) returning all five user records

Figure 11: UNION-based payload extracting usernames and password hashes for all accounts

Figure 12: Brute Force module login form

Figure 13: Failed login attempt showing 'Username and/or password incorrect'

Figure 14: Failed attempt with visible GET request parameters in the browser URL bar

Figure 15: Successful login test (admin/password), used alongside Figure 14 to confirm no lockout or delay on either outcome

Figure 16: Successful login confirmation message

Figure 17: VirtualBox error encountered when attempting to boot the Kali VM for automated brute-force testing

Figure 18: Lab architecture diagram

Executive Summary

This engagement tested a deliberately vulnerable web application in an isolated lab to demonstrate how raw penetration testing results are converted into an accountable remediation process, not just a list of problems.

- SQL Injection (Critical) — an attacker with no credentials can extract the entire user database, including password hashes, in a single request.
- OS Command Injection (Critical) — an attacker can run arbitrary operating system commands through the application's ping feature.
- No Account Lockout (High) — the login form allows unlimited guesses with no delay, lockout, or CAPTCHA, and transmits credentials insecurely.

Overall, the application's current security posture is poor and would not withstand even an unsophisticated attacker. The two Critical findings in particular require action within days, not weeks, and every finding in this report has been assigned an owner and a fixed deadline to ensure that doesn't just stay true on paper.

Chapter 1 — Introduction

1.1 Background

Red teaming is a security exercise where I, acting as an ethical attacker, use the same tactics, techniques and procedures a real attacker would use — not just to list what's broken, but to understand how weaknesses connect into a real attack path. A basic vulnerability scan tells you a door is unlocked. Red teaming tells you what happens once someone walks through it, and whether anyone notices.

For this project, I wasn't trying to prove I could break something for its own sake. The point was to prove I could run a small, honest security test from start to finish, and then do the harder part most beginner projects skip: explain what the findings actually mean for a business, assign someone to fix them, and track that fix through to a verified close. I built a controlled lab using the Damn Vulnerable Web Application (DVWA), ran real attacks against it, and carried the results through a full debrief-and-remediation lifecycle, the way a professional engagement would.

This report also reflects a course requirement worth stating plainly: all testing in this project stayed inside an isolated, instructor-approved lab environment running on my own machine. No real, unauthorised, or third-party system was scanned, probed, or touched at any point.

1.2 Problem Statement

Many security assessments stop at identifying vulnerabilities, but organizations often fail to convert findings into measurable remediation actions. This project develops a post-engagement debrief and remediation tracking report to improve accountability, prioritization, and closure of red team findings.

1.3 Aim

The aim of this project is to demonstrate, through a controlled lab engagement, how raw red team findings are converted into a structured, accountable post-engagement debrief and remediation tracking framework that ensures verified risk reduction rather than a static list of bugs.

1.4 Scope and Limitations

Testing was carried out against a local instance of DVWA, deployed via the official `vulnerables/web-dvwa` Docker image and mapped to port 80 on my machine. The container's MariaDB database and Apache web server both started successfully, timestamped Fri Jul 17 15:10:35 2026 in the container logs (Figure 4). I logged in using the default credentials (`admin/password`) and deliberately set the application's internal security level to Low — this switches off DVWA's built-in protections so each vulnerability is visible in its rawest, most demonstrable form, which was a conscious choice made to expose the maximum attack surface, not an oversight (Figure 7).

A few honest limitations shaped this project. The target scope was a single application inside an isolated lab, not a full enterprise network or a live Active Directory environment, so findings here should be read as demonstrations of vulnerability classes rather than a full enterprise risk

assessment. Testing relied on manual exploitation rather than an automated enterprise scanner (such as Nessus or Qualys), partly due to environment constraints and partly because manual testing gave a far more granular understanding of exactly how each exploit worked, which matters more for a learning-focused engagement than scanner coverage would. Every activity stayed inside the local container loopback interface — no external scanning, no lateral movement, and no data left the lab.

One constraint is worth calling out specifically, because it shaped Finding RT-03's methodology. My original plan was to automate the brute-force test with Hydra from a Kali Linux VM. That VM failed to boot, throwing an Unresolved (unknown) host platform error (VERR_UNRESOLVED_ERROR, Result Code E_FAIL) in VirtualBox — almost certainly caused by Docker Desktop's Hyper-V backend and VirtualBox competing for the same CPU virtualisation extensions on the same Windows machine (Figure 17). Given the time available, I made a deliberate engineering decision to drop the automated approach and test the same weakness manually instead. That trade-off, and why it doesn't weaken the finding, is explained fully in Section 4.3.

Chapter 2 — Objectives

This project is guided by six core objectives:

1. Analyse the results of a controlled red team lab engagement within a containerised environment.
2. Document attack paths and specific control failures identified during active exploitation.
3. Conduct a structured post-engagement debrief that bridges raw exploit data and defensive analysis.
4. Build a remediation tracking framework that translates identified vulnerabilities into actionable operational tasks.
5. Prioritise remediation actions based on a balanced assessment of severity, exploitability, and business impact.
6. Define a formal retesting and closure workflow so implemented fixes are verified before risks are archived.

Chapter 3 — Literature and Tooling Review

This project's structure follows established industry standards for how professional security assessments are run and reported, rather than an improvised format. This chapter also maps the engagement's findings against MITRE ATT&CK, the industry-standard reference for attacker behaviour, to ground the technical work in a recognised framework rather than informal description.

3.1 Penetration Testing Execution Standard (PTES)

PTES covers the full lifecycle of an assessment, from scoping to post-exploitation reporting. A core reporting principle within PTES is splitting deliverables into a high-level, business-facing Executive Summary and a detailed, practitioner-facing Technical Report. This project follows that same split — the Executive Summary above and this technical body are kept deliberately distinct, aimed at different readers with different needs and different levels of technical fluency.

3.2 OWASP Open Penetration Test Reporting Standard (OPTRS)

OPTRS emphasises findings that plug directly into a development team's existing workflow — not just a description of a flaw, but enough context for a developer to fix it without guesswork. The tracking matrix in Chapter 7 follows this same philosophy: each finding is broken into clear, reusable fields (Asset, Severity, Owner, Due Date, Status) that could be dropped straight into a bug tracker such as Jira or GitHub Issues.

3.3 NIST SP 800-61 Revision 3

NIST SP 800-61r3 stresses that organisations should formally review and learn from any major security incident or exercise, not just patch and move on. That principle is the justification for Chapter 6's debrief content going beyond a simple bug list into the actual root cause behind each flaw — why it exists, not just that it exists.

3.4 CISA Remediation Guidance

CISA's guidance is direct: a vulnerability isn't actually being managed unless it has a named owner, a real deadline, and an interim mitigation plan. That standard is exactly what the remediation tracker in Chapter 7 is built to meet — every finding in this report carries an owner and a due date, not just a description.

3.5 MITRE ATT&CK Mapping

MITRE ATT&CK is a publicly maintained knowledge base of real-world attacker tactics and techniques, widely used across the industry as a common vocabulary for describing what an attacker actually did, rather than a vendor-specific label. Mapping each finding to a specific technique ID also makes this report easier for a blue team to act on, since detection and monitoring tooling (SIEM rules, EDR alerts) is very often built directly around ATT&CK technique IDs.

Finding	MITRE ATT&CK Technique	ID
SQL Injection (RT-01)	Exploit Public-Facing Application / Unsecured Credentials	T1190 / T1552
OS Command Injection (RT-02)	Command and Scripting Interpreter	T1059
No Account Lockout (RT-03)	Brute Force — Credential Guessing	T1110.001

Framing the findings this way also clarifies what a defender should actually watch for: repeated 500/408-style errors or unusual SQL syntax in web server logs for T1190, unexpected child processes spawned by the web server for T1059, and a high volume of failed authentication requests from a single source for T1110.001.

Chapter 4 — Methodology

4.1 Lab Environment Setup

The assessment environment was built in Docker to keep it fully isolated and reproducible. The target was the official vulnerables/web-dvwa container (Figures 1-3), establishing a standard DVWA

instance. The application's security level was deliberately set to Low (Figure 7), switching off input sanitisation, regex filtering, and any WAF-style protection, so each vulnerability could be observed in its raw, unmitigated form. Testing was carried out on 17 July 2026, with the container confirmed live from 15:10:35 that afternoon (Figure 4). Figure 18 below shows the overall lab topology: the attacker browser on the Windows host communicates over HTTP with the DVWA container, which separates its PHP/Apache application layer from its MariaDB data layer, with both writing to container logs used later as evidence.

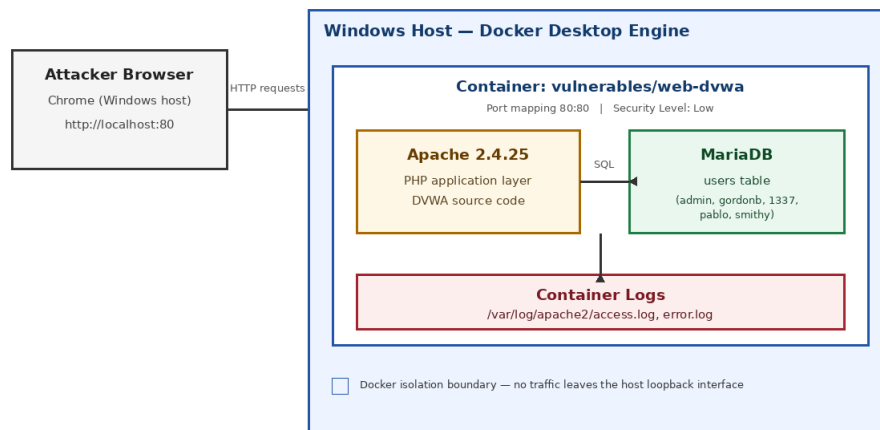


Figure 18: Lab architecture diagram

4.2 Engagement Phases

The project followed a structured, six-stage process:

[1. Scope Definition] → [2. Reconnaissance] → [3. Exploitation] → [4. Validation] → [5. Debrief] → [6. Tracking Design]

- **Scope Definition:** the assessment was restricted exclusively to the local DVWA deployment — nothing outside the container.
- **Reconnaissance:** the exposed attack surface was mapped by walking through DVWA's navigation, identifying the SQL Injection, Command Injection, and Brute Force modules as targets.
- **Exploitation and Testing:** specific strings were manually injected into input forms and URL parameters to evaluate input handling and state changes; some attempts failed and some succeeded, and both outcomes informed the findings.
- **Finding Validation:** every successful exploit condition was re-tested under identical parameters to verify repeatability and rule out false positives before documentation.
- **Debrief:** an architectural analysis traced each validated finding back to its underlying root cause and control failure.
- **Remediation Tracking Design:** validated technical data was converted into an operational framework, assigning severity, owners, and fixed deadlines.

4.3 Tools and Techniques Used

SQL Injection.

Testing began with a normal baseline request — User ID 1 — which correctly returned a single user record (Figure 9), confirming the application was behaving normally before exploitation began. Submitting `1' OR '1'='1' -- -` into the same field returned all five users in the database instead of one (Figure 10), confirming the input was not sanitised and the query's logic had been hijacked. Escalating with `1' UNION SELECT user, password FROM users -- -` extracted the full username and password-hash pairs for every account, including `admin:5f4dcc3b5aa765d61d8327deb882cf99` (Figure 11). Notably, `admin` and a second account, `smithy`, shared the identical hash — proof the application uses unsalted MD5, meaning identical passwords always produce identical hashes and are trivially crackable via rainbow tables.

OS Command Injection.

The 'ping a device' module was tested by submitting `127.0.0.1 && whoami` instead of a plain IP address. The application returned the expected ping output followed by `www-data` on its own line (Figure 8) — proof that a second, attacker-supplied command executed on the server. The `www-data` result also confirms the web server runs as a low-privilege user rather than root, a detail that shapes the severity assessment honestly rather than overstating it as a full system compromise.

Authentication / Brute Force.

The original plan was to automate this test with Hydra from a Kali Linux VM. That plan hit a real, verifiable wall: the Kali VM failed to boot, throwing an Unresolved (unknown) host platform error (`VERR_UNRESOLVED_ERROR`, Result Code `E_FAIL`) in VirtualBox (Figure 17) — almost certainly caused by Docker Desktop's Hyper-V backend and VirtualBox competing for the same CPU virtualisation extensions on the same Windows machine. Given the time remaining, a deliberate engineering decision was made to drop the automated approach and test the same weakness manually. Server access logs confirmed a successful login (`admin/password`) processed as a plain GET request, with the password visible directly in the URL (Figure 15, and see the raw log excerpt in Appendix B). A deliberately incorrect submission (`admin/wrongpass1`) was tested separately and confirmed the same behaviour for a failed attempt: still a GET request, still no CAPTCHA, still no lockout or delay before another guess was accepted (Figures 13-14). These two tests were run at different points in the session rather than back-to-back, but that does not weaken the finding — what matters is that at no point, successful or failed, did DVWA ever throttle, delay, or block a login attempt. A real attacker doesn't need speed to exploit this; they need patience, and the application will never stop them.

Why this matters.

- SQL Injection shows how unsanitised input, a well-known and critical vulnerability class, leads directly to a full data breach.
- Command Injection shows how the same root cause — trusting user input — escalates past the database and into the operating system itself.

- The missing lockout shows that even without a clever exploit, weak authentication design by itself is a real, exploitable risk.

Chapter 5 — Implementation

This chapter documents exactly what was built, configured, and deployed to produce the findings in Chapter 6 — the practical steps behind the methodology described in Chapter 4.

5.1 Container Deployment

The lab was brought up with a single Docker command: `docker run --rm -it -p 80:80 vulnerables/web-dvwa`. The `--rm` flag ensures no residual container state persists between runs, keeping the environment reproducible for repeated testing. The `-p 80:80` flag maps the container's internal port 80 directly to the host's port 80, which is why the application was reachable at `http://localhost` with no additional network configuration. Docker pulled six image layers on first run (Figure 2) before starting two services inside the container: MariaDB (the database server) and Apache (the web server), both confirmed healthy in the startup logs (Figure 3, Figure 4).

5.2 Application Configuration

DVWA exposes its vulnerability behaviour through a single configuration control: the Security Level setting, accessible from the DVWA Security page and stored server-side for the session. Setting this to Low (Figure 7) is what actually enabled every exploit in this report — at Medium or High settings, DVWA applies partial input filtering (such as basic keyword blacklisting or type-casting on the ID parameter) that would have blocked or complicated the exact payloads used in Chapter 6. Choosing Low was therefore not a way of making the application artificially easy; it was a way of isolating the underlying vulnerability class from DVWA's optional teaching scaffolding, so the root cause could be demonstrated cleanly.

5.3 Payload Construction

Each payload used in this engagement was built incrementally, starting from a known-good baseline and changing one variable at a time, so that any change in application behaviour could be attributed to the exact input that caused it. For SQL Injection, that meant starting from the numeric baseline (`id=1`), then introducing a single quote to test for unescaped input, then building up to a full boolean OR condition and finally a UNION-based extraction. For Command Injection, the baseline was a valid IP address, with a shell metacharacter (`&&`) appended to test whether the input reached a shell interpreter. For the brute-force test, the same login form was used for both a known-incorrect and a known-correct credential pair, isolating the authentication logic itself as the variable under test rather than the payload.

5.4 Evidence Capture

Every finding in this report is backed by at least one of two evidence types: a browser screenshot showing the application's response, and — where available — the corresponding line from the container's Apache access log, extracted directly from the running container's log stream rather than reconstructed afterward. Using both sources matters: a browser screenshot can be staged, but a server-side access log entry showing the exact request method, path, and query string is much

harder to fabricate convincingly, which is why the brute-force finding in particular leans on the access log (Appendix B) rather than the screenshots alone.

5.5 Testing Timeline

The table below reconstructs the actual sequence of testing from container logs and screenshot timestamps, rather than an idealised order. This is included deliberately: Section 4.3 already explains why the brute-force test's two outcomes were captured out of sequence, and this timeline makes that transparent rather than glossing over it.

- 15:10 — Container confirmed live (MariaDB and Apache both healthy).
- 15:12-15:15 — DVWA login, home page, and Security Level review; security level set to Low.
- 15:15 — SQL Injection baseline request (ID: 1).
- 15:16-15:17 — Docker/Apache startup logs reviewed and captured for documentation.
- 15:18 — Successful brute-force login (admin/password) recorded in the access log.
- 15:19 — Browser confirmation of the successful login captured.
- 15:34-15:36 — SQL Injection OR and UNION payloads executed and captured.
- 15:40 — Command Injection payload executed and captured.
- 15:36-16:04 — Failed brute-force attempt (admin/wrongpass1) tested and captured, including the URL bar view.
- 15:56 — Kali VM boot failure encountered and captured while preparing the (ultimately abandoned) automated brute-force approach.

Chapter 6 — Results and Findings

6.1 Finding RT-01: SQL Injection — Logic Bypass & Credential Exfiltration

Asset: DVWA SQL Injection module (/vulnerabilities/sqli/) | Severity: Critical | ATT&CK: T1190, T1552

Description: The User ID input field passes user-supplied data directly into a SQL query without sanitisation or parameterisation, allowing an attacker to alter the query's logic entirely.

Evidence: A baseline request (Figure 9) returned a single expected user. Submitting `1' OR '1'='1' -- -` (Figure 10) returned all five user records instead of one. A follow-up UNION-based payload, `1' UNION SELECT user, password FROM users -- -` (Figure 11), extracted full username and password-hash pairs for every account, including `admin:5f4dcc3b5aa765d61d8327deb882cf99` and `gordonb:e99a18c428cb38d5f260853678922e03`.

Business Impact: An attacker with no valid credentials can enumerate the entire user base and extract every stored password hash in a single request, enabling full account takeover once hashes are cracked offline.

Root Cause: No parameterised queries or input sanitisation — user input is concatenated directly into SQL statements. Compounded by unsalted MD5 password hashing: `admin` and `smithy` produced identical hashes in the evidence above, making them trivially crackable via rainbow tables.

Recommendation: Implement parameterised queries/prepared statements throughout. Migrate password storage to a salted, adaptive hashing algorithm (bcrypt or Argon2). Enforce least-privilege database accounts for the web application.

Owner: Development/Database Team | Due: 3 days (Critical) | Status: Open | Retest: Pending

6.2 Finding RT-02: OS Command Injection — Remote Code Execution

Asset: DVWA Command Injection module (/vulnerabilities/exec/) | Severity: Critical | ATT&CK: T1059

Description: The 'ping a device' feature passes user input directly to the underlying operating system shell without validation.

Evidence: Submitting `127.0.0.1 && whoami` (Figure 8) returned the expected ping statistics followed by `www-data`, confirming the injected command executed successfully.

Business Impact: An attacker can execute arbitrary OS commands with the privileges of the web server process, enabling reconnaissance, file access, or lateral movement. Confirmed impact here is limited to the low-privilege `www-data` account rather than root.

Root Cause: User-supplied input is passed to a shell execution function without sanitisation, allowing, or a safe API alternative.

Recommendation: Eliminate direct shell invocation for user-influenced input; use language-level networking libraries instead of shelling out. Where shell use is unavoidable, strictly validate input against an allow-list and use parameterised command execution (e.g. `escapshellarg`).

Owner: Development Team | Due: 3 days (Critical) | Status: Open | Retest: Pending

6.3 Finding RT-03: Insufficient Authentication Controls — No Account Lockout

Asset: DVWA Brute Force module (/vulnerabilities/brute/) | Severity: High | ATT&CK: T1110.001

Description: The login form accepts unlimited authentication attempts with no lockout, delay, rate-limiting, or CAPTCHA after failed logins. Credentials are also transmitted via GET request parameters, exposing them in browser history and server logs.

Evidence: A failed attempt (admin/wrongpass1) returned 'Username and/or password incorrect' (Figure 13), with the credentials visible directly in the browser's URL bar (Figure 14). A successful login (admin/password) was separately confirmed via server access logs and browser evidence (Figures 15-16, Appendix B), also transmitted via GET. No lockout, delay, or CAPTCHA was triggered on either outcome, at any point in testing.

Business Impact: The absence of lockout or throttling permits unattended, automated credential-guessing attacks against any account without triggering a defensive response. Credential exposure via GET parameters compounds this risk should logs or browser history be accessed by a third party.

Root Cause: No server-side attempt-counting or lockout logic is implemented; authentication is handled via GET rather than POST.

Recommendation: Implement account lockout or exponential back-off after a defined threshold of failed attempts. Switch credential submission to POST. Add a CAPTCHA after repeated failures. Log and alert on repeated authentication failures from a single source.

Owner: Security/Development Team | Due: 7 days (High) | Status: Open | Retest: Pending

6.4 Summary of Findings

The three findings above differ in mechanism but share a common thread, developed further in Chapter 8: each one exists because the application trusted input it should have validated, whether that input was a database query parameter, a shell command argument, or a login attempt.

- RT-01 (Critical): unauthenticated, single-request full credential exfiltration via SQL Injection.
- RT-02 (Critical): unauthenticated arbitrary command execution at web-server privilege via OS Command Injection.
- RT-03 (High): unlimited, undetected credential guessing via missing authentication controls.

Chapter 7 — Remediation Tracking Framework

7.1 Tracker Structure

The tracker below exists to close the gap between finding a problem and actually fixing it. In line with the CISA and OWASP principles covered in Chapter 3, a finding is only useful if it carries a specific owner and a real deadline — otherwise it sits in a backlog indefinitely. Every row below has both, plus a status field so each finding's progress from discovery to resolution stays visible and auditable.

7.2 The Tracker Table

ID	Finding	Asset	Severity	Owner	Due	Status	Retest
RT-01	SQL Injection — logic bypass & credential exfiltration	DVWA SQLi module	Critical	Dev/DB Team	3 days	Open	Pending
RT-02	OS Command Injection (RCE)	DVWA Command Injection module	Critical	Dev Team	3 days	Open	Pending
RT-03	No account lockout on login	DVWA Brute Force module	High	Security/Dev Team	7 days	Open	Pending

7.3 Prioritization Logic

RT-01 and RT-02 sit at Critical severity with a 3-day window because both give an unauthenticated attacker direct, serious compromise capability with minimal effort — RT-01 exposes the full user database and every password hash, and RT-02 grants arbitrary command execution on the host. RT-03 is High rather than Critical, with a 7-day window: a missing lockout is a genuine structural failure, but exploiting it relies on a sustained guessing campaign rather than a single request, earning it slightly more time to fix without being deprioritised.

7.4 Retest and Closure Workflow

Open → In Progress → Mitigated → Retest Pending → Closed

A finding only moves to Closed once dedicated retesting formally confirms the fix neutralises the vulnerability, using the same conditions, parameters, and payloads originally used to identify the flaw.

Chapter 8 — Analysis and Recommendations

8.1 Cross-Finding Analysis

Taken individually, each finding in this report is a distinct vulnerability class. Taken together, they point to one underlying pattern: input arriving from outside the application — whether through a form field, a URL parameter, or a login attempt — is trusted by default rather than validated by default. SQL Injection and Command Injection are, at the code level, the same mistake expressed in two different execution contexts: a database query and an operating system shell. The missing account lockout is a different category of failure — not an input-handling bug but a missing control — yet it shares the same root posture: the application assumes good faith on the part of whoever is submitting a request.

This matters for prioritisation beyond the individual severity ratings in Chapter 7. A organisation seeing all three findings together should read this as a signal that the application's entire input-handling discipline needs review, not just three isolated tickets. Patching RT-01 and RT-02 without addressing the underlying pattern (e.g. a secure coding standard, mandatory code review for anything touching a query or a shell call, and a dependency on vetted libraries rather than hand-rolled string concatenation) would likely leave a fourth, undiscovered instance of the same mistake elsewhere in the codebase.

8.2 Severity Reasoning

Severity in this report was assigned using a simplified likelihood-times-impact model consistent with how CVSS-style scoring reasons about a vulnerability, without requiring a full CVSS vector calculation for a single-application lab. RT-01 and RT-02 both score high on likelihood (no authentication required, low attacker skill needed, payload delivered in a single request) and high on impact (full data exposure; arbitrary command execution), which is why both land at Critical. RT-03 scores lower on likelihood in isolation — a successful guess is not instant — but its impact, once a guess succeeds, is identical to a legitimate login, and the complete absence of any defensive control (no lockout, no delay, no CAPTCHA, no logging-based alerting observed) is itself unusual enough to warrant High rather than Medium severity.

8.3 Broader Recommendations

Beyond the finding-specific fixes in Chapter 6, three broader changes would meaningfully reduce this application's overall exposure:

- Adopt a web application firewall (WAF) or reverse proxy with basic input-pattern filtering as a compensating control while the underlying code fixes in Chapter 6 are implemented — this buys time without removing the need for a real code-level fix.
- Introduce centralised, alertable logging (a SIEM or even a simple log-forwarding pipeline) so that patterns like repeated failed logins from a single source, or unusual SQL syntax in request parameters, trigger an alert rather than sitting unnoticed in a log file, which is exactly what happened in this lab until the log was reviewed manually.

- Build these three vulnerability classes into the organisation's secure code review checklist and onboarding material for new developers, since all three stem from well-documented, well-known mistake patterns that a checklist-driven review would catch before code reaches production.

8.4 Limitations of This Analysis

This analysis is scoped to what a single-application, single-session lab can actually support. The severity reasoning in Section 8.2 is a simplified model, not a formal CVSS calculation, and would need to be redone against the organisation's real environment (network exposure, data sensitivity, compensating controls already in place) before being used to justify budget or resourcing decisions. Similarly, the cross-finding pattern identified in Section 8.1 is drawn from three findings in one application; a genuine secure-coding-gap conclusion for an organisation would need this same exercise repeated across multiple applications before being treated as more than a reasonable hypothesis.

Chapter 9 — Conclusion and Recommendations

This project ran a simulated adversarial engagement against an isolated DVWA instance, identifying three high-impact vulnerabilities and carrying all three through to a structured, production-style remediation tracker rather than stopping at a list of bugs.

The core argument of this work is simple: vulnerability identification generates no inherent business value on its own. A security assessment only produces a measurable defensive return once its raw technical findings are systematically converted into owned, timed, and empirically validated remediation work. A report that lists flaws and stops there hasn't reduced anyone's risk — it has only moved the paperwork.

Collectively, the three findings documented here — SQL injection, command injection, and a lack of brute-force throttling — illustrate a broader industry reality: basic input validation gaps and weak authentication hardening remain the primary compromise vectors for real web applications, even without anything exotic in the attack technique itself. Each of these required nothing more than trusting user input that should never have been trusted.

Looking forward, as assessments scale up to larger environments — full corporate networks or live Active Directory infrastructure — the same discipline demonstrated here would still hold: the post-engagement debrief and structured tracking accountability matter as much as the exploitation itself, because that is the part that actually converts into verified risk reduction.

References

Cybersecurity and Infrastructure Security Agency. (n.d.). Vulnerability remediation guidance. U.S. Department of Homeland Security. <https://www.cisa.gov/>

diginja. (n.d.). DVWA: Damn Vulnerable Web Application [Computer software]. GitHub. <https://github.com/digininja/DVWA>

MITRE Corporation. (n.d.). MITRE ATT&CK framework. <https://attack.mitre.org/>

National Institute of Standards and Technology. (2025). NIST SP 800-61 Revision 3: Incident response recommendations and considerations for cybersecurity risk management. U.S. Department of Commerce. <https://nvlpubs.nist.gov/nistpubs/specialpublications/nist.sp.800-61r3.pdf>

OWASP Foundation. (n.d.). Open Penetration Test Reporting Standard (OPTRS). <https://owasp.org/www-project-penetration-test-reporting-standard/>

The Penetration Testing Execution Standard. (n.d.). Reporting. <http://www.pentest-standard.org/index.php/Reporting>

Appendix A — Evidence Screenshots

All figures below are original evidence, captured during the lab engagement.

Environment Setup

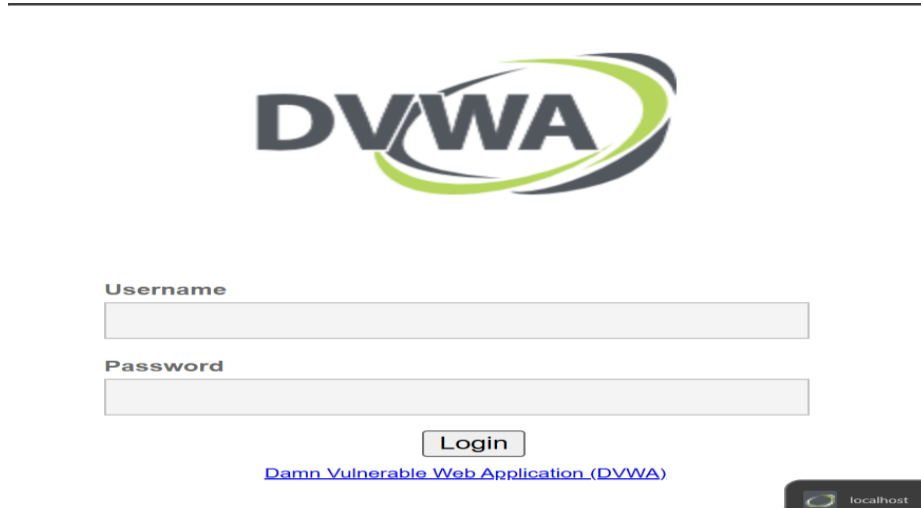


Figure 1: Docker version check and initial container pull command

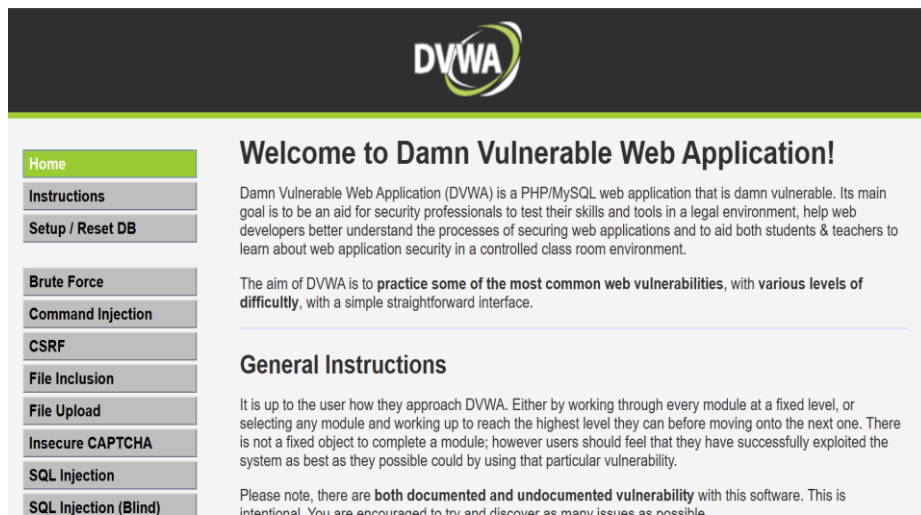


Figure 2: Docker image layers pulling successfully

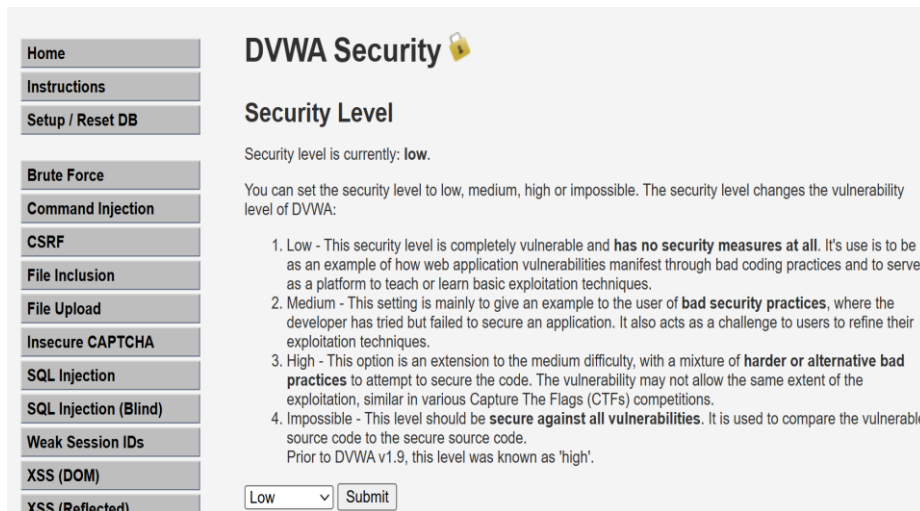


Figure 3: MariaDB and Apache services starting inside the container

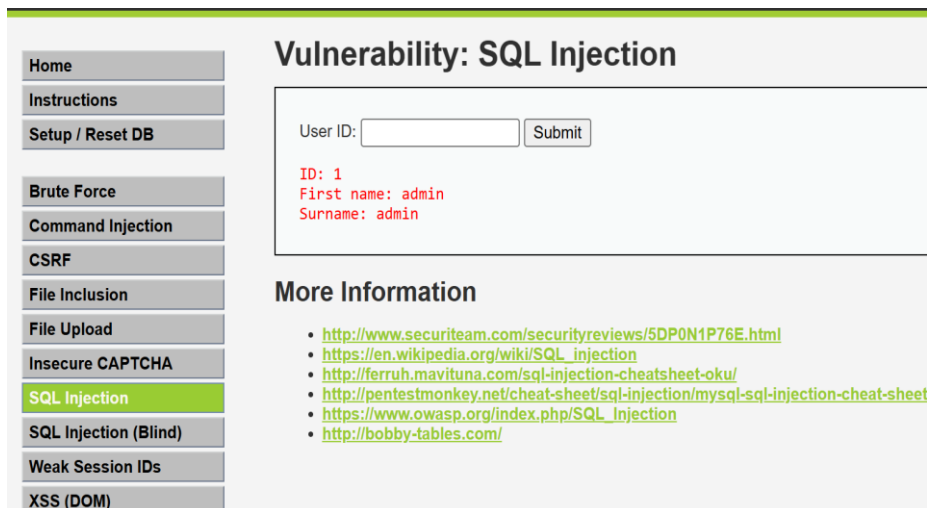


Figure 4: Apache server logs confirming successful startup

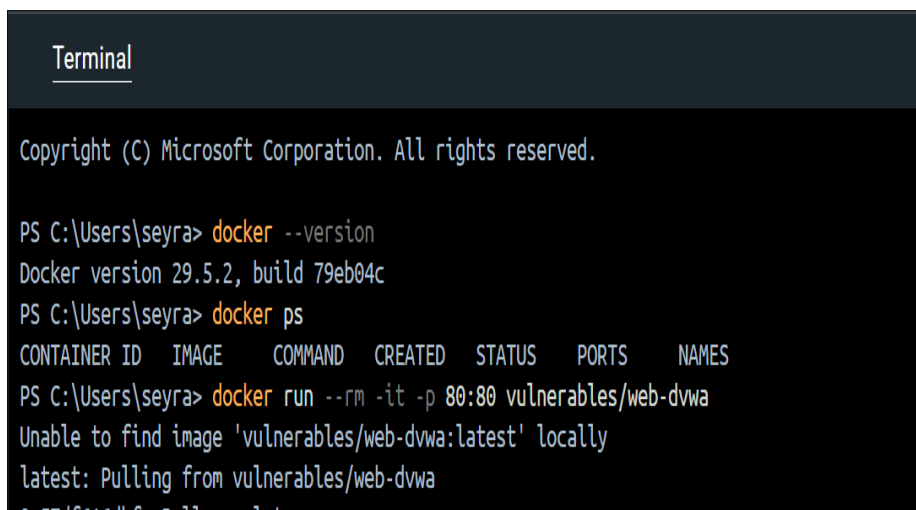


Figure 5: DVWA login page

```

Terminal

CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
PS C:\Users\seyra> docker run --rm -it -p 80:80 vulnerables/web-dvwa
Unable to find image 'vulnerables/web-dvwa:latest' locally
latest: Pulling from vulnerables/web-dvwa
0c57df616dbf: Pull complete
3e17c6eae66c: Pull complete
eb05d18be401: Pull complete
b3d64a33242d: Pull complete
e9968e5981d2: Pull complete
6cff5f35147f: Pull complete

```

Figure 6: DVWA home page describing the application's purpose

```

Terminal

2cd72dba8257: Pull complete
Digest: sha256:dae203fe11646a86937bf04db0079adef295f426da68a92b40e3b181f337daa7
Status: Downloaded newer image for vulnerables/web-dvwa:latest
[+] Starting mysql...
[ ok ] Starting MariaDB database server: mysqld.
[+] Starting apache
[....] Starting Apache httpd web server: apache2AH00558: apache2: Could not reliably det
ermine the server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName'
directive globally to suppress this message
ok

```

Figure 7: DVWA Security Level page set to Low

Finding RT-02: Command Injection

```

Terminal

directive globally to suppress this message
. ok
==> /var/log/apache2/access.log <==

==> /var/log/apache2/error.log <==
[Fri Jul 17 15:10:35.727532 2026] [mpm_prefork:notice] [pid 300] AH00163: Apache/2.4.25
(Debian) configured -- resuming normal operations
[Fri Jul 17 15:10:35.727703 2026] [core:notice] [pid 300] AH00094: Command line: '/usr/s
bin/apache2'

```

Figure 8: Command injection payload (127.0.0.1 && whoami) returning ping output plus www-data

Finding RT-01: SQL Injection

```
Terminal

781 "http://localhost/vulnerabilities/sqli/?id=1&Submit=Submit" "Mozilla/5.0 (Windows NT
10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
"

172.17.0.1 - - [17/Jul/2026:15:16:50 +0000] "-" 408 0 "-" "-"

172.17.0.1 - - [17/Jul/2026:15:18:07 +0000] "GET /vulnerabilities/brute/?username=admin&
password=password&Login=Login HTTP/1.1" 200 1822 "http://localhost/vulnerabilities/brute
/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chr
ome/150.0.0.0 Safari/537.36"

172.17.0.1 - - [17/Jul/2026:15:18:07 +0000] "GET /hackable/users/admin.jpg HTTP/1.1" 200
3829 "http://localhost/vulnerabilities/brute/?username=admin&password=password&Login=Lo
```

Figure 9: Baseline SQL query — normal input (ID: 1) returning single expected user

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)

Vulnerability: SQL Injection

User ID:

ID: 1' OR '1'='1' -- -
First name: admin
Surname: admin

ID: 1' OR '1'='1' -- -
First name: Gordon
Surname: Brown

ID: 1' OR '1'='1' -- -
First name: Hack
Surname: Me

ID: 1' OR '1'='1' -- -
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1' -- -
First name: Bob
Surname: Smith

Figure 10: Logic bypass payload (1' OR '1'='1' -- -) returning all five user records

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass

Vulnerability: SQL Injection

User ID:

ID: 1' UNION SELECT user, password FROM users -- -
First name: admin
Surname: admin

ID: 1' UNION SELECT user, password FROM users -- -
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' UNION SELECT user, password FROM users -- -
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

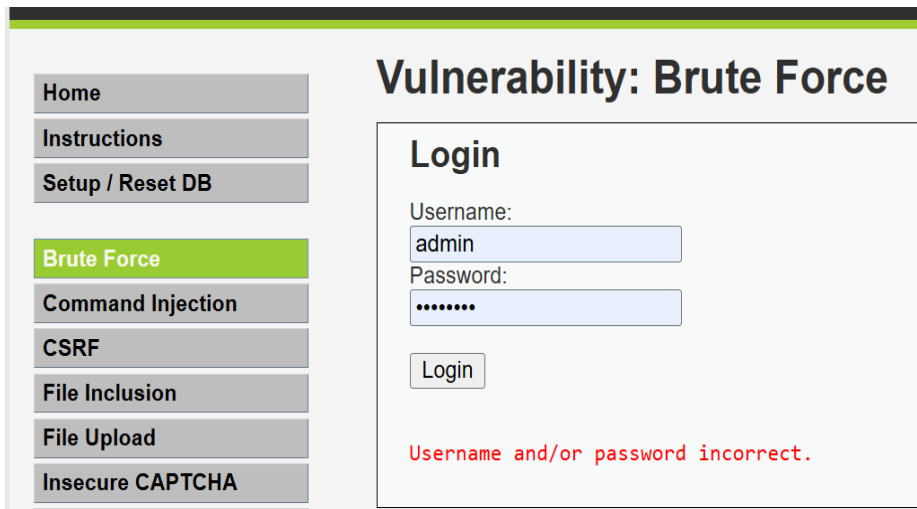
ID: 1' UNION SELECT user, password FROM users -- -
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1' UNION SELECT user, password FROM users -- -
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1' UNION SELECT user, password FROM users -- -
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

Figure 11: UNION-based payload extracting usernames and password hashes for all accounts

Finding RT-03: Brute Force / Account Lockout



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

Vulnerability: Brute Force

Login

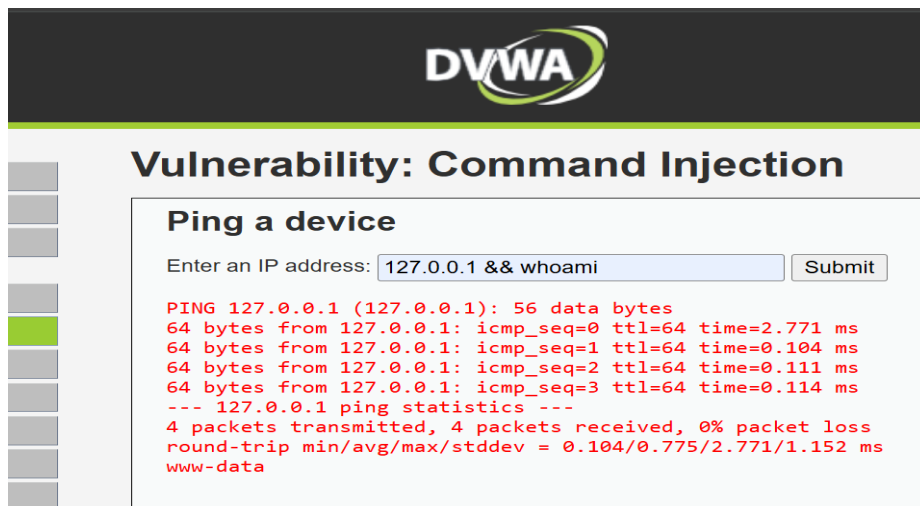
Username:
admin

Password:
.....

Login

Username and/or password incorrect.

Figure 12: Brute Force module login form



DVWA

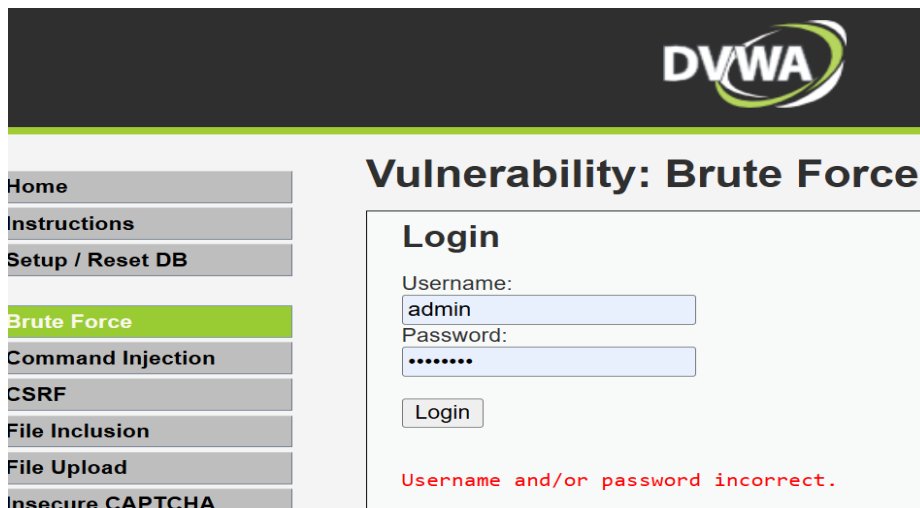
Vulnerability: Command Injection

Ping a device

Enter an IP address: 127.0.0.1 && whoami Submit

PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=2.771 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.104 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.111 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.114 ms
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.104/0.775/2.771/1.152 ms
www-data

Figure 13: Failed login attempt showing 'Username and/or password incorrect'



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

Vulnerability: Brute Force

Login

Username:
admin

Password:
.....

Login

Username and/or password incorrect.

Figure 14: Failed attempt with visible GET request parameters in the browser URL bar

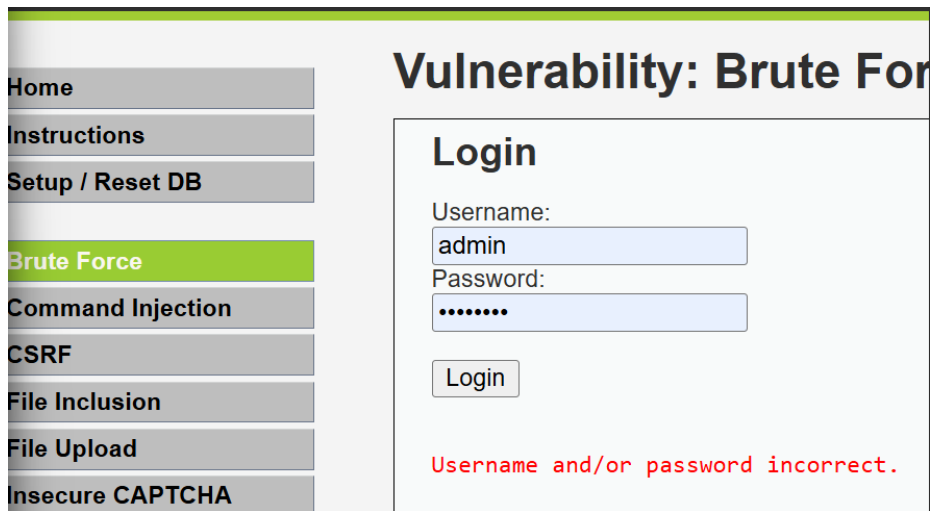


Figure 15: Successful login test (admin/password), used alongside Figure 14 to confirm no lockout or delay on either outcome

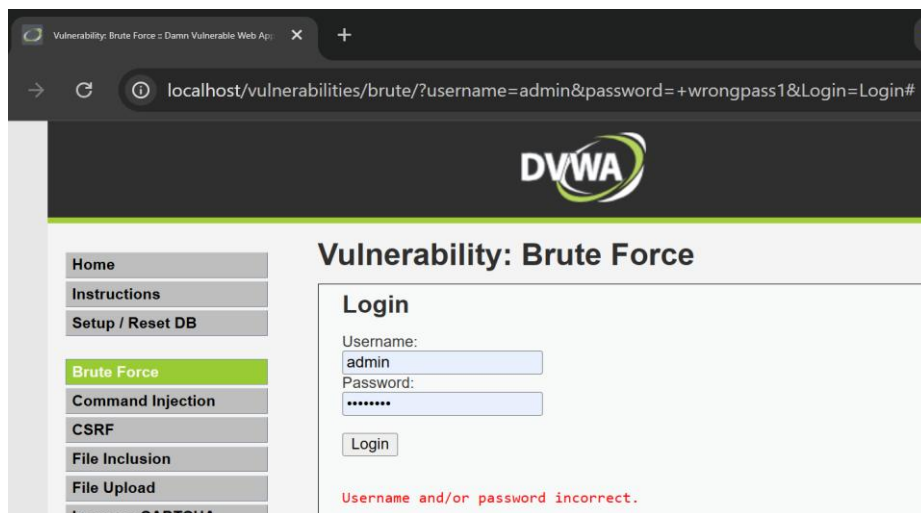


Figure 16: Successful login confirmation message

Methodology Constraint: Kali VM / Hydra

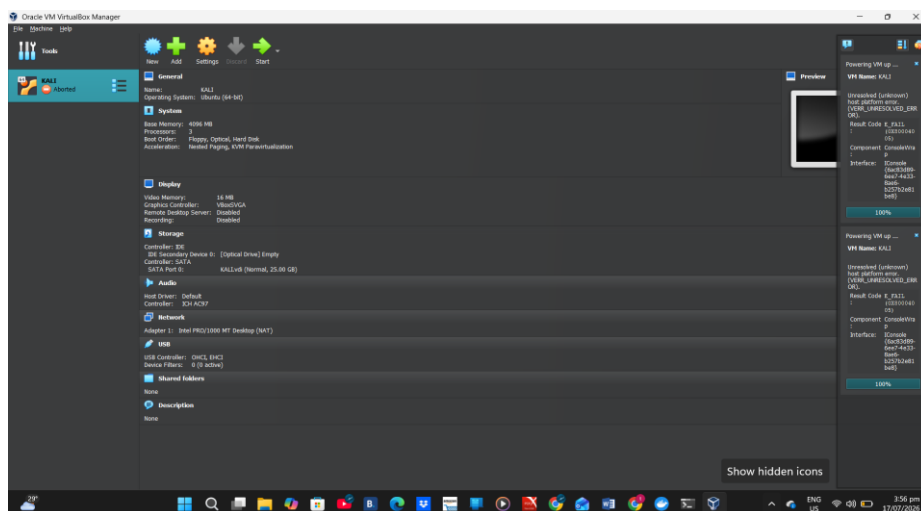


Figure 17: VirtualBox error (VERR_UNRESOLVED_ERROR, Result Code E_FAIL) encountered when attempting to boot the Kali VM for automated brute-force testing

Appendix B — Raw Log Excerpts

Raw excerpts from the DVWA container's Apache access log (/var/log/apache2/access.log), included to independently corroborate the browser-based evidence in Appendix A.

```
172.17.0.1 - - [17/Jul/2026:15:16:50 +0000] "-" 408 0 "-" "-" 172.17.0.1 - -  
[17/Jul/2026:15:18:07 +0000] "GET  
/vulnerabilities/brute/?username=admin&password=password&Login=Login HTTP/1.1" 200 1822  
"http://localhost/vulnerabilities/brute/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64)  
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36" 172.17.0.1 - -  
[17/Jul/2026:15:18:07 +0000] "GET /hackable/users/admin.jpg HTTP/1.1" 200 3829  
"http://localhost/vulnerabilities/brute/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64)  
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36"
```

This excerpt confirms two things independently of any screenshot: first, that the successful brute-force login was processed as an HTTP GET request with the password sitting in plain text in the query string; and second, the exact server-side timestamp (15:18:07) of that request, used in Section 4.3 to establish the actual sequence of testing.